ATAL BIHARI VAJPAYEE - INDIAN INSTITUTE OF INFORMATION TECHNOLOGY AND MANAGEMENT, GWALIOR

PENTAELLIPTIC CURVE CRYPTOGRAPHY (PEECC) BASE IMPLEMENTATION AND IMPROVEMENTS OF UNDERWATER SENSOR NETWORKS

CYG Course Project Report

Supervisor: Dr. Vijaypal Singh Rathore

Department of Information Technology, ABV-IIITM Gwalior

Prasun Baranwal (2023IMT-060)
Shivam Deolankar (2023IMT-073)

Problem Statement

UWCN authentication protocols secures underwater communication using mutual authentication and PEECC, but implementing and verifying its pentatope-based arithmetic and computational performance remains challenging.

- **Resource-Constrained Environment:** Underwater acoustic networks face **limited bandwidth, high latency, packet loss, and restricted energy resources**, making **efficient cryptographic operations essential**.

- **Dependence on PEECC:** The protocol is based on pentatope-based elliptic curve cryptography for secure key generation and data exchange, but the **underlying encryption scheme requires further implementation-level investigation.**

- **Missing Implementation Details:** The base paper omits **message-to-point mapping**, **explicit curve group laws**, and a reproducible implementation, complicating independent verification of encryption and decryption.

- **Verification Limitations:** Protocol-level verification does not guarantee the security of the underlying cryptographic implementation.

Produce a possible implementation of PEECC in C using known methods and curves secp256r1 and GMP to verify correctness, measure performance, and investigate potential security weaknesses.

Literature Review and Code Base

Primary Reference

A Novel and Robust Authentication Protocol for Secure Underwater Communication Systems

Secondary Reference

Pentatope Based Elliptic Curve Encryption for Privacy and Protection of Multimedia Data

Codebase

https://github.com/D3athSkulll/Auth-Protocol-for-Secure-UWCN

Objectives

Primary

Implement PEECC [18] in C with GMP on secp256r1, then the full UWC authentication protocol (M1-M12) of the IoT paper on top of it.

A tested, measured and attackable implementation for correctness, performance and security.

Secondary

Key generation, encryption and decryption with the paper's formulas; Koblitz message-to-point mapping and multi-block messages.

Fix the weakness: hardened mode (random pentatope input, point validation, compressed points).

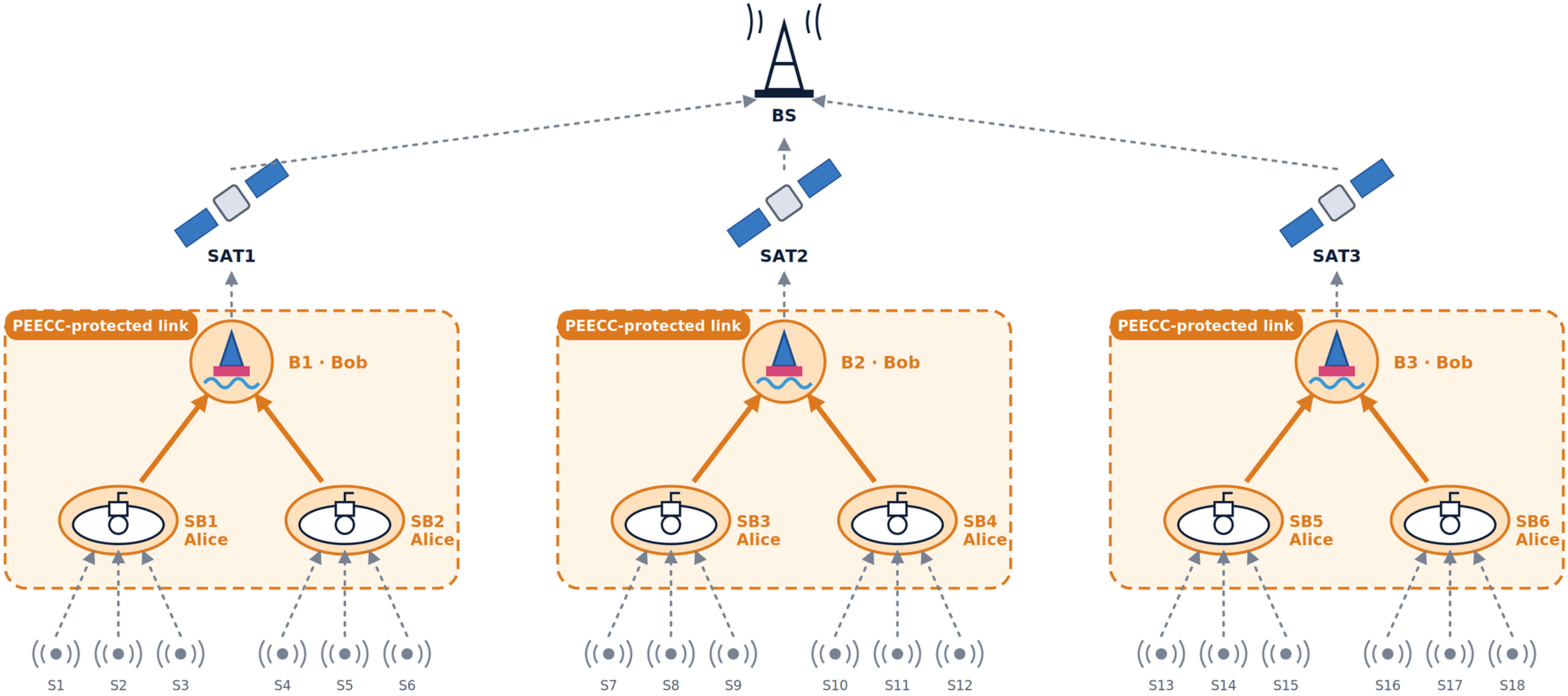
Technical

27 self-tests, 1,000 random round trips, 11 protocol attacks and a Scyther model.

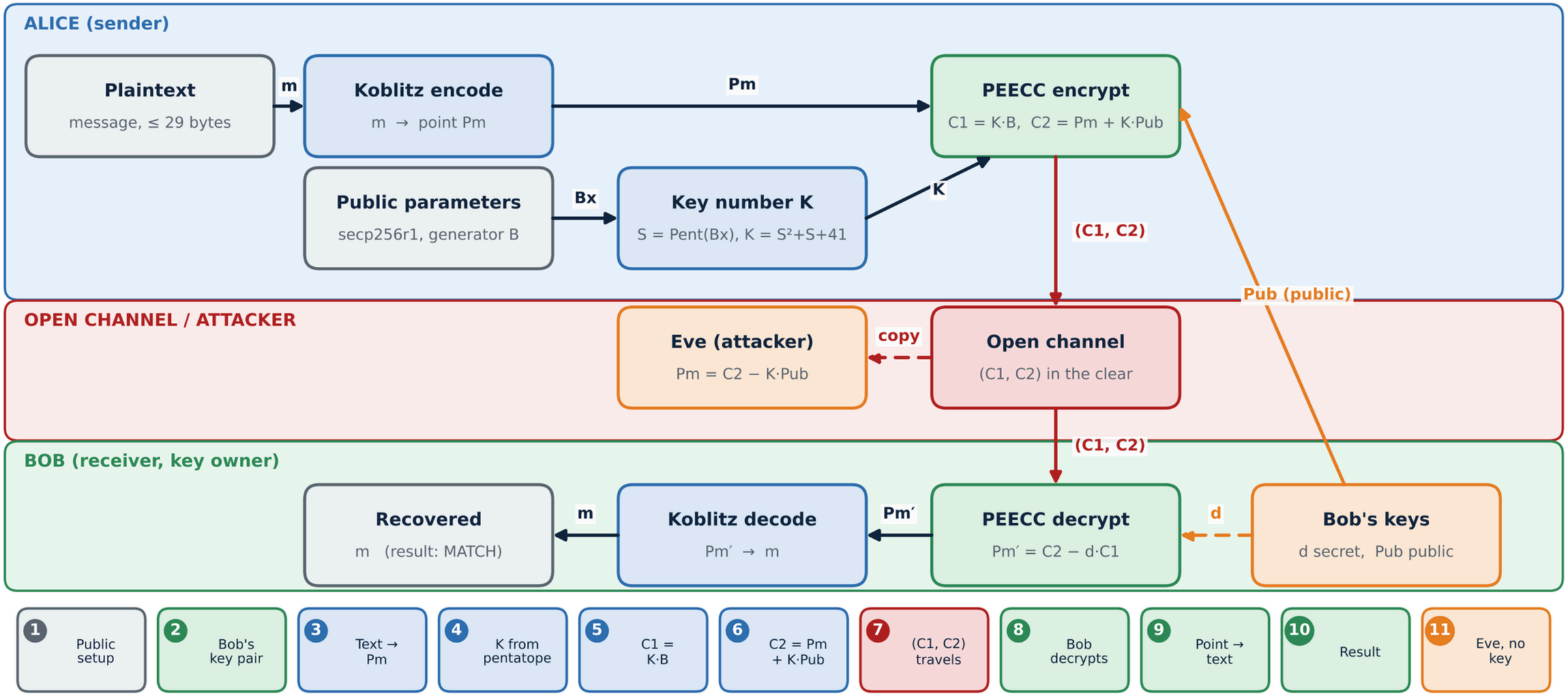
Measure computation, communication, energy and latency, and compare every paper claim with the measured value.

BASE PEECC Use Case

Sensor data is relayed upward through submarines, buoys and satellites to the base station.



BASE PEECC PIPELINE



Parameters, Library and Curve

PEECC parameters

- **Prime p:** 256-bit prime; all coordinates are computed **mod p**.
- **Curve:** $y^2 = x^3 + ax + b \pmod p$, $a = p - 3$, b fixed (secp256r1).
- **Generator B** of prime order **N**; private keys in [1, N-1].
- **B_x**, the x-coordinate of B, feeds the pentatope number in the paper.
- **S** = $x(x+1)(x+2)(x+3)/24$ (pentatope), **K** = S^2+S+41 (Euler).

Why GMP?

- Arbitrary precision for the **2,029-bit K**; fast modular arithmetic; native C API.

Why secp256r1?

- Standard published constants (SEC 2), 256-bit prime field; results can be checked against known values (2·B).

Why a 2-D curve?

- The IoT paper's 5-D curve ψ has no arithmetic; [18] defines PEECC on an ordinary curve.

Algorithm Overview

[A] Setup

- **Keys:** d from OS randomness, Pub = d*B (double-and-add).
- **Encode:** m = 0x01 || bytes, x = 256m + j until x^3+ax+b is a square (Koblitz).
- **Key number:** S = Pent(x), K = S^2+S+41 with x = B_x (paper) or **x random (hardened)**.

[B] Encryption at Alice

- C1 = K*B, C2 = Pm + K*Pub; send (C1, C2), **compressed in hardened mode**.
- **Paper: K comes from public B_x, so anyone can compute it.**

[C] Decryption at Bob

- **Hardened:** reject C1, C2 that are off the curve.
- Pm = C2 - d*C1
- = Pm + K*Pub - d*K*B
- = Pm + K*Pub - K*(d*B)
- = Pm + ~~K*Pub - K*Pub~~
- Decode: m = floor(x / 256), check and drop the 0x01 marker.

Messages > 29 bytes

- Split into blocks; one (C1, C2) per block.

Pentatope Euler and Key Value K

```c
static void pentatope(mpz_t s, const mpz_t x) {
    mpz_t t; mpz_init(t);
    mpz_add_ui(t, x, 1); mpz_mul(s, x, t);
    mpz_add_ui(t, x, 2); mpz_mul(s, s, t);
    mpz_add_ui(t, x, 3); mpz_mul(s, s, t);
    mpz_divexact_ui(s, s, 24);
    mpz_clear(t);
}

/* [18] Sec. 3.2 Step 3: K = S^2 + S + 41. */
static void euler_K(mpz_t k, const mpz_t s) {
    mpz_mul(k, s, s); mpz_add(k, k, s); mpz_add_ui(k, k, 41);
}
```

peecc.c → pentatope() and euler_K()

Pentatope number

$S = x(x+1)(x+2)(x+3) / 24 = C(x+3, 4)$
the division by 24 is always exact

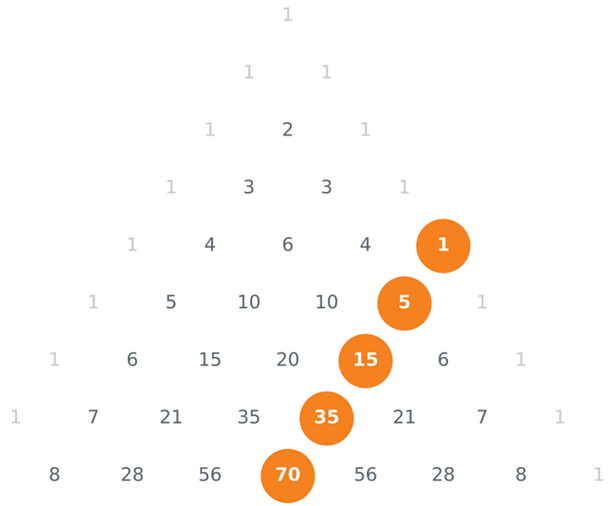
Euler polynomial

$K = S^2 + S + 41$
x = Bx (generator x-coordinate), all plain integers

The implementation reproduces the pentatope-based calculations using arbitrary-precision arithmetic to handle extremely large integer values.

The full value of (K) follows the original paper's construction but increases computational cost and introduces a confidentiality weakness because it is publicly computable.

Pascal's triangle



orange diagonal: C(x+3, 4) = 1, 5, 15, 35, 70, ...

x	S	K = S² + S + 41
1	1	43
2	5	71
3	15	281

hand values used by the self-tests

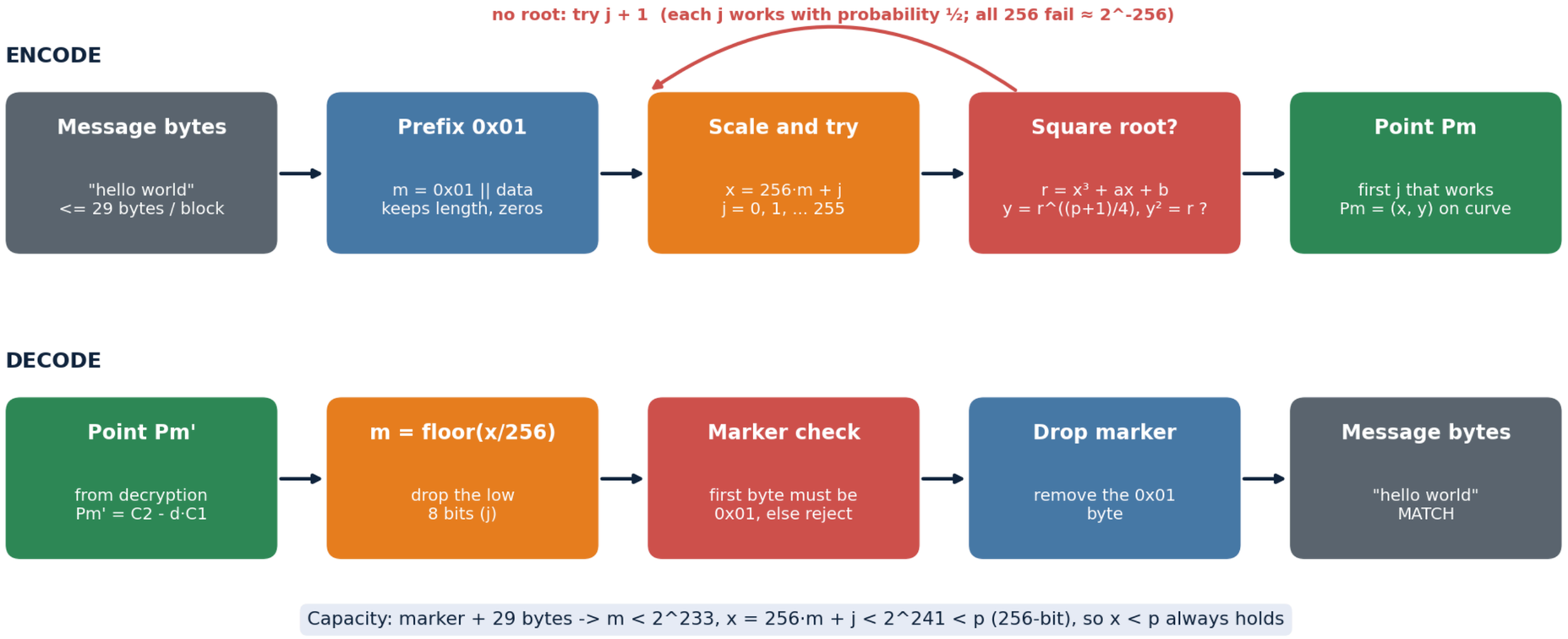
Real generator: S ≈ 2^1015, K ≈ 2^2029

Point Multiplication and Koblitz Mapping

peecc.c -> pt_mul(): double-and-add

```c
/* Double-and-add scalar multiplication r = k * q (scan k from its top bit down). */
static void pt_mul(Pt *r, const mpz_t k, const Pt *q) {
    Pt acc, base; pt_init(&acc); pt_init(&base); pt_set(&base, q);
    for (long i = (long)mpz_sizeinbase(k, 2) - 1; i >= 0; i--) {
        pt_double(&acc, &acc);
        if (mpz_tstbit(k, i)) pt_add(&acc, &acc, &base);
    }
    pt_set(r, &acc); pt_clear(&acc); pt_clear(&base);
}
```

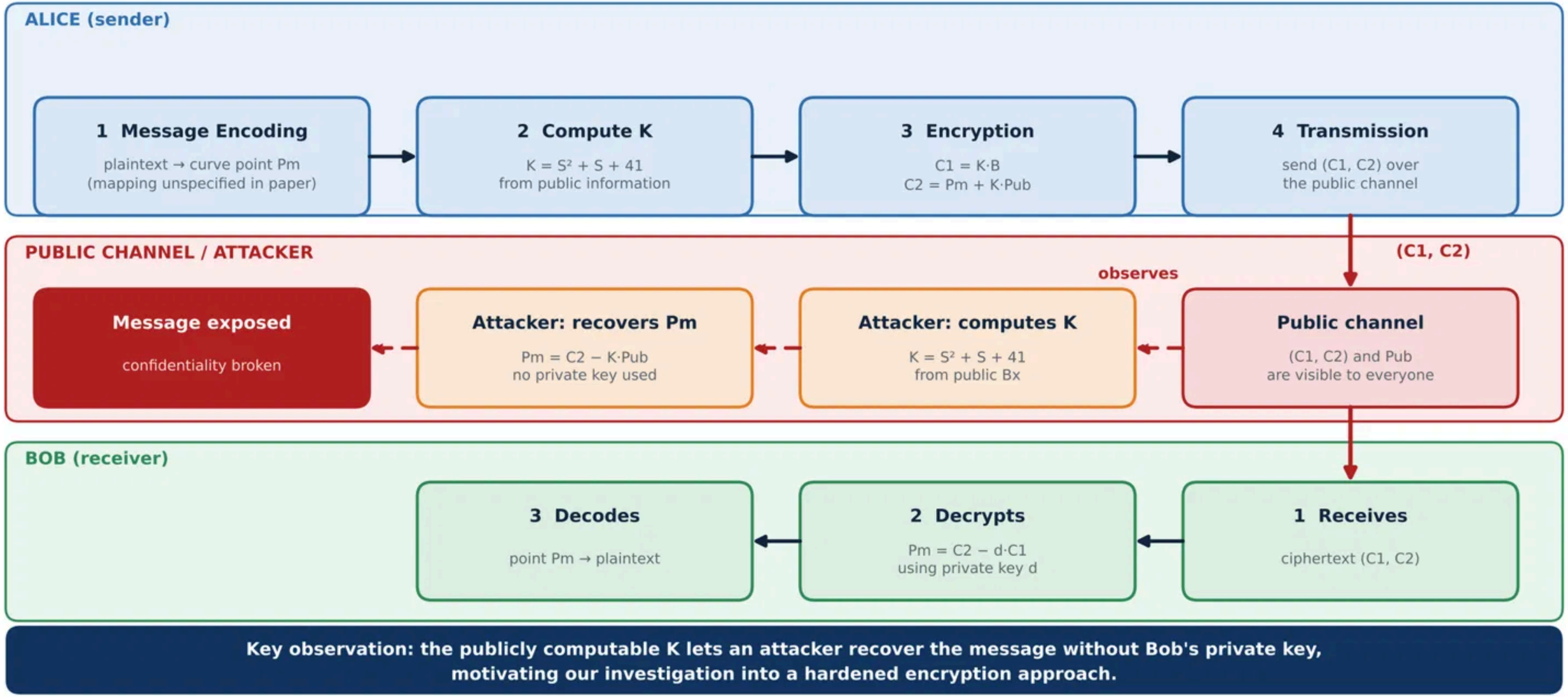
- Start at infinity; per bit of k **double**, and **add** the base point if the bit is 1.
- 256-bit scalar: ~256 doublings + 128 additions; paper's 2029-bit K: ~3065 operations.



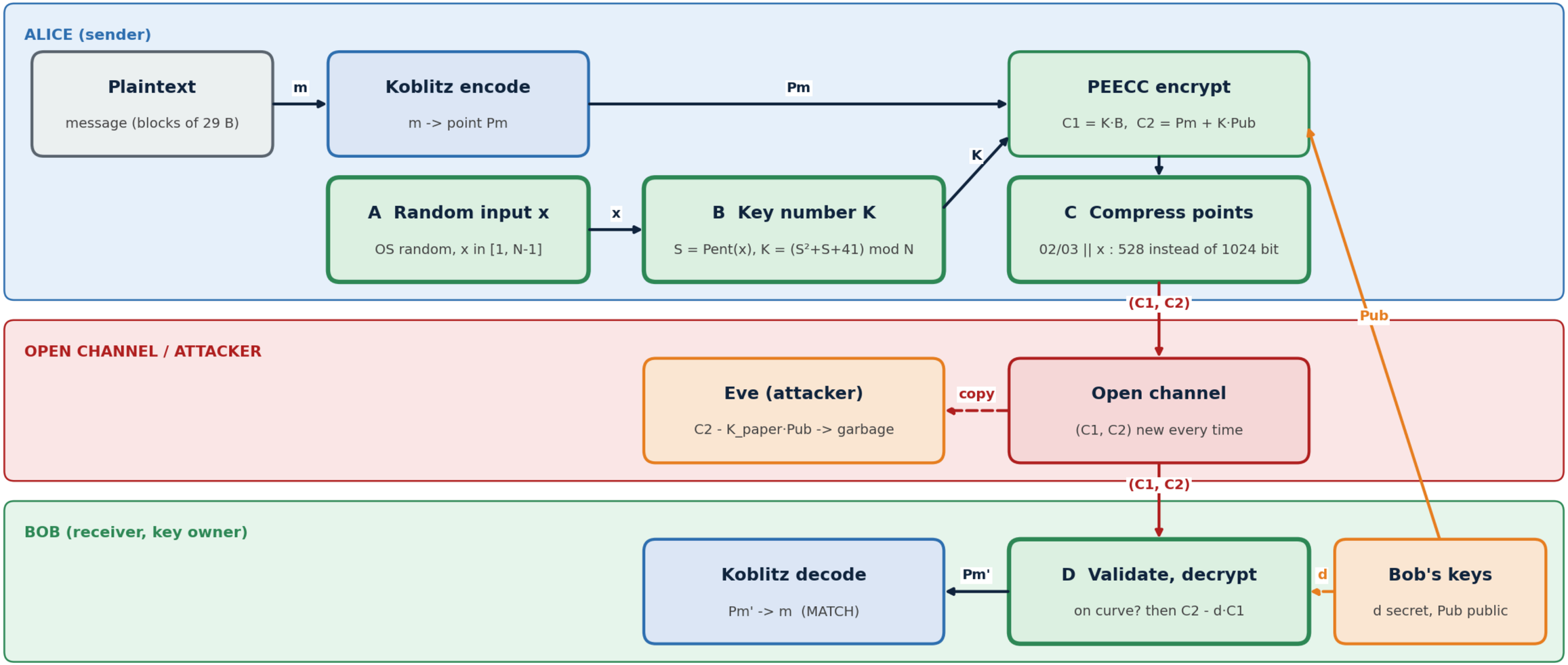
Capacity: marker + 29 bytes -> m < 2^233, x = 256·m + j < 2^241 < p (256-bit), so x < p always holds

BASE PEECC Flaw

The original PEECC scheme enables Alice to encrypt messages for Bob,
but its publicly computable encryption value compromises confidentiality.



IMPROVED PEECC PIPELINE



A + B: pentatope and Euler steps kept, only the input changes (B_x -> secret random x) **C: half the bits on the wire** **D: forged / off-curve points rejected**

Improved PEECC in Code

```c
static void make_K(mpz_t k, mpz_t x, mpz_t s, int hardened) {
    do {
        if (hardened) random_private_key(x); else mpz_set(x, G.x);
        pentatope(s, x); euler_K(k, s);
        if (hardened) mpz_mod(k, k, N);
    } while (mpz_sgn(k) == 0);
}
static void encrypt_K(Pt *c1, Pt *c2, const Pt *pm, const Pt *pub, const mpz_t k) {
    Pt t; pt_init(&t);
    pt_mul(c1, k, &G);
    pt_mul(&t, k, pub);
    pt_add(c2, pm, &t);
    pt_clear(&t);
}
static int encrypt(Pt *c1, Pt *c2, const Pt *pm, const Pt *pub, int hardened) {
    if (hardened && !pt_valid(pub)) return 0;
    mpz_t k, x, s; mpz_init(k); mpz_init(x); mpz_init(s);
    make_K(k, x, s, hardened);
    encrypt_K(c1, c2, pm, pub, k);
    mpz_clear(k); mpz_clear(x); mpz_clear(s);
    return 1;
}
static int decrypt(Pt *pm, const Pt *c1, const Pt *c2, const mpz_t d, int hardened) {
    if (hardened && !(pt_valid(c1) && pt_valid(c2))) return 0;
    Pt t; pt_init(&t);
    pt_mul(&t, d, c1); pt_neg(&t, &t);
    pt_add(pm, c2, &t);
    pt_clear(&t);
    return 1;
}
```

```c
static int pt_from_cbytes(Pt *q, const uint8_t *in) {
    int zero = 1;
    for (int i = 0; i < CPT_BYTES; i++) zero &= in[i] == 0;
    if (zero) { q->inf = 1; return 1; }
    if (in[0] != 2 && in[0] != 3) return 0;
    mpz_import(q->x, 32, 1, 1, 1, 0, in + 1);
    if (mpz_cmp(q->x, P) >= 0) return 0;
    mpz_t rhs, e, chk; mpz_init(rhs); mpz_init(e); mpz_init(chk);
    fp_mul(rhs, q->x, q->x); fp_add(rhs, rhs, A); fp_mul(rhs, rhs, q->x); fp_add(rhs, rhs, Bc);
    mpz_add_ui(e, P, 1); mpz_fdiv_q_2exp(e, e, 2);
    mpz_powm(q->y, rhs, e, P);
    fp_mul(chk, q->y, q->y);
    int ok = mpz_cmp(chk, rhs) == 0;
    if (ok && (mpz_odd_p(q->y) != 0) != (in[0] == 3)) mpz_sub(q->y, P, q->y);
    q->inf = 0;
    mpz_clear(rhs); mpz_clear(e); mpz_clear(chk);
    return ok;
}
```

- **make_K:** x = B_x (paper) or **secret random x**; K reduced mod N.
- **pt_valid:** off-curve C1, C2, Pub rejected.
- **pt_from_cbytes:** 02/03 || x, 1024 -> **528 bits** per block.

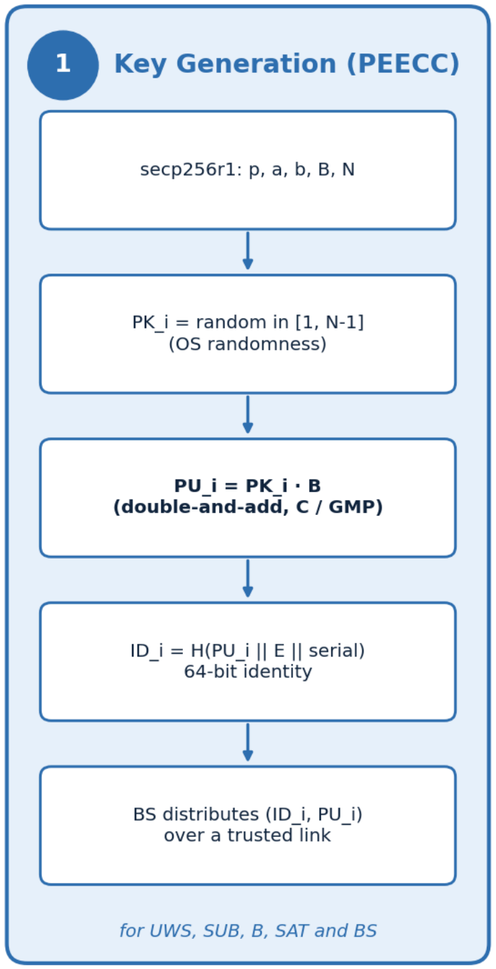
UWC Authentication Protocol (M1-M12)

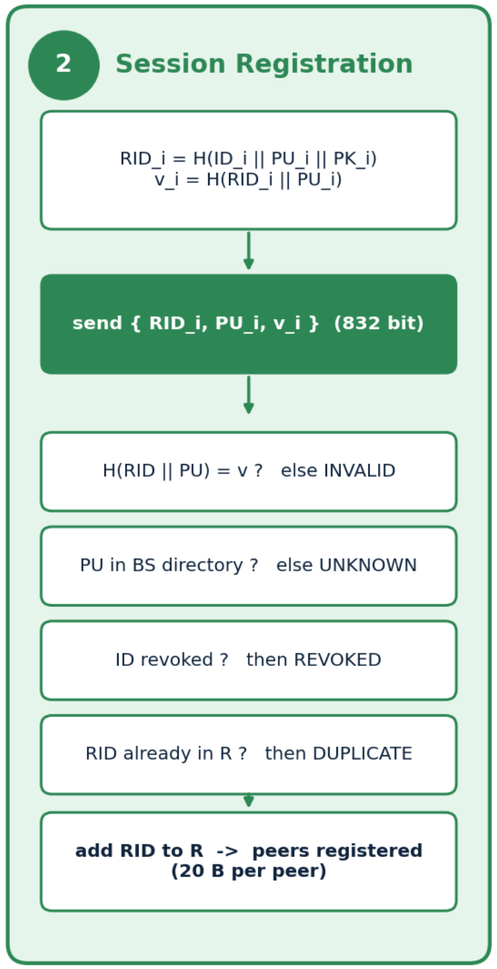


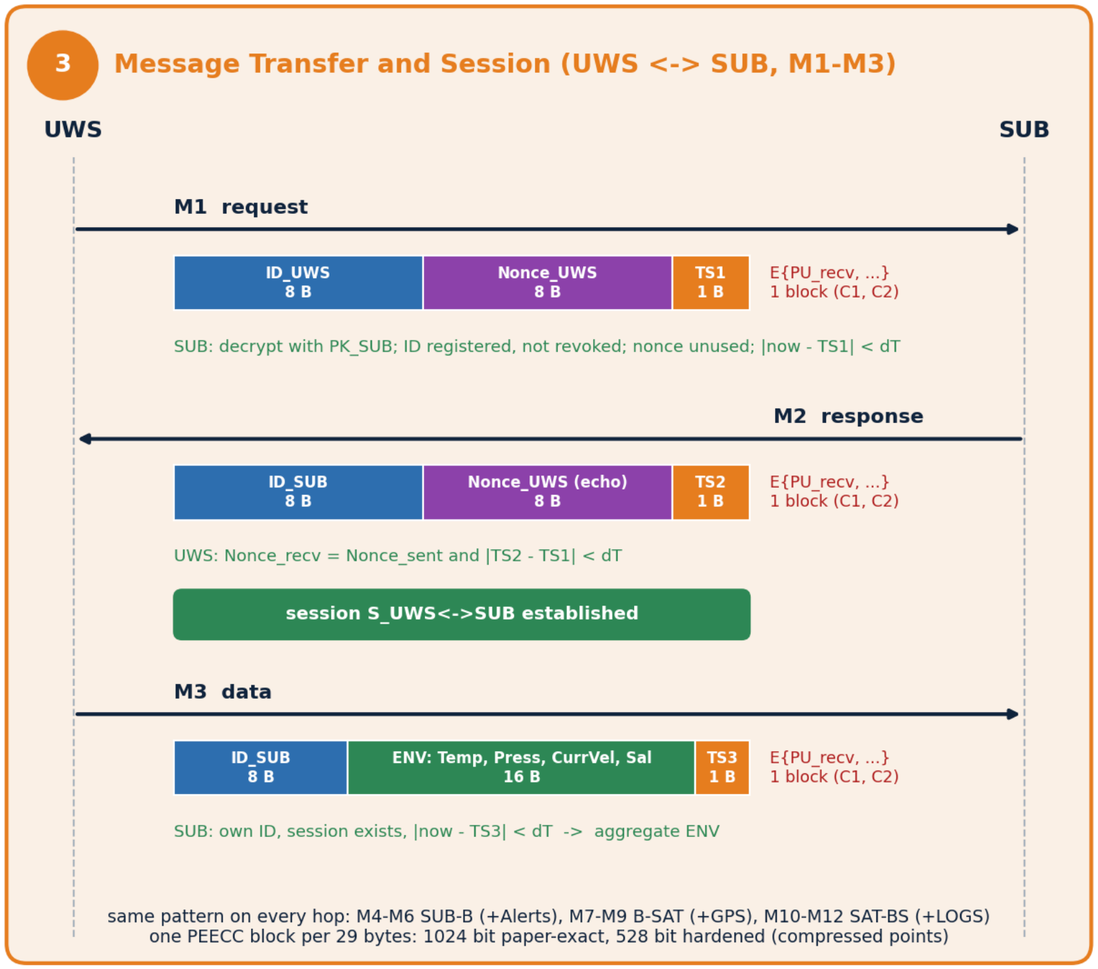
Every message is PEECC-encrypted with the receiver's public key. Blue: handshake (ID, nonce, 8-bit TS). Green: data (ENV, Alerts, GPS, LOGS).

protocol.py (IoT Sec. IV): initialization ID = H(PU||E||serial), registration RID / v with INVALID / DUPLICATE / REVOKED checks, M1-M12 with nonce and adaptive ΔT checks, fallback (B1 -> B2, SAT1 -> SAT2), revocation list; all curve arithmetic in C through ctypes (peecc.dll).

PEECC-Based UWCN: Three Phases







Every entity first generates its PEECC key pair, then registers with its neighbours; only registered, unrevoked peers can open a session, and every message M1-M12 is PEECC-encrypted with the receiver's public key.

Output - peecc.exe (hello world)

```
$ ./peecc.exe "hello world"
mode: paper-exact [18] (x = Bx, K is public)
== Key generation (receiver) [18] Sec. 3.1 ==
  private key d          62eb6b0a2638e00ae8caef1ba95dd84fd6b484...
      x = d313eb9994a3d9d61796d13fce372e1b6bd6a74c44fd9bdce1415...
== Encryption [18] Sec. 3.2 ==
  message: "hello world" (11 bytes)
  message point Pm (Koblitz)
      x = 168656c6c6f20776f726c6401
  x = Bx (public)        6b17d1f2e12c4247f8bce6e563a440f277037d...
  S = Pentatope(x)       53a0e0a9de034f682e2c4fa21cea25263c18fe...
  K = S^2+S+41           (2029 bits)
  C1 = K*B
      x = 681dc23639d030c4df03de37d6459fc1c8bde47e06c619c2c40d1...
  C2 = Pm + K*Pub
      x = 897aac298746c2c917809b49fa22a3021f34046c628ba42aa90a7...
== Decryption [18] Sec. 3.3 ==
  Pm' = C2 - d*C1
      x = 168656c6c6f20776f726c6401
  decoded message: "hello world"
  result: MATCH
== Keyless eavesdropper: C2 - K_paper*Pub, no private key ==
  eavesdropper reads: hello world
  attack: SUCCEEDS (message not confidential)
```

```
$ ./peecc.exe --hardened "hello world"
mode: HARDENED (deviation from [18]: random pentatope input x p...
== Key generation (receiver) [18] Sec. 3.1 ==
  private key d          b7cbc2de0f454d8caec1883763752220d47f4b...
      x = 50dfce7582c7952679d7374158b7c58d0c3bf590858e82fe72166...
== Encryption [18] Sec. 3.2 ==
  message: "hello world" (11 bytes)
  message point Pm (Koblitz)
      x = 168656c6c6f20776f726c6401
  x = random (secret)    756d97a7e8a629e7d7b6a1eef88f69f47f5841...
  S = Pentatope(x)       78e44deded62df9eceaf903714469ed5619b08...
  K = (S^2+S+41) mod N   (255 bits)
  C1 = K*B
      x = 65e546817bef7023060829debf57294c2f0ff76363390cbdc962b...
  C2 = Pm + K*Pub
      x = 387358e81ae148c96d4a0d997ac47b8499a4fea306154e17734b5...
== Decryption [18] Sec. 3.3 ==
  Pm' = C2 - d*C1
      x = 168656c6c6f20776f726c6401
  decoded message: "hello world"
  result: MATCH
== Keyless eavesdropper: C2 - K_paper*Pub, no private key ==
  eavesdropper reads: (garbage, decoding fails)
  attack: FAILS
```

- **Paper-exact (left):** K has 2029 bits, C1 is always 681dc236..., and a keyless eavesdropper **reads "hello world"**.
- **Hardened (right):** K has 255 bits, decryption still MATCHES, and the eavesdropper **gets garbage**.
- **make test:** 27 of 27 checks pass (curve, keys, pentatope, 1000 round trips, K-public break, hardened mode, blocks, compression).

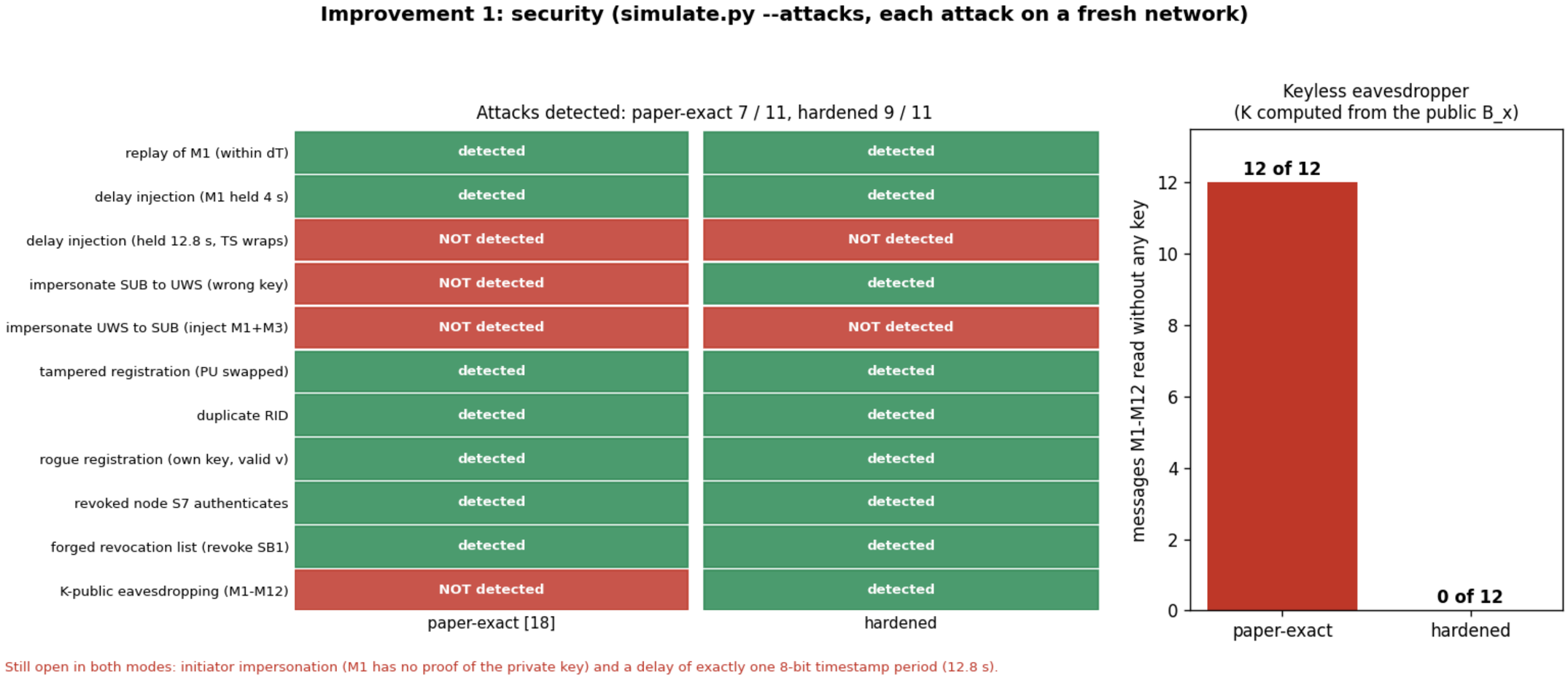
Output - Simulation and Scyther

```
$ python simulate.py --attacks
attack                                      paper-exact   hardened
replay of M1 (within dT)                    detected      detected
delay injection (M1 held 4 s)               detected      detected
delay injection (held 12.8 s, TS wraps)     NOT DETECTED  NOT DETECTED
impersonate SUB to UWS (wrong key)          NOT DETECTED  detected
impersonate UWS to SUB (inject M1+M3)       NOT DETECTED  NOT DETECTED
tampered registration (PU swapped)          detected      detected
duplicate RID                               detected      detected
rogue registration (own key, valid v)       detected      detected
revoked node S7 authenticates               detected      detected
forged revocation list (revoke SB1)         detected      detected
K-public eavesdropping (M1-M12)             NOT DETECTED  detected
```



- **simulate.py --attacks:** 11 attacks on fresh networks; hardened mode adds detection of SUB impersonation and K-public eavesdropping.
- **Scyther (--max-runs=5): 23 Ok, 35 Fail.** Responders cannot authenticate the initiator, since M1 has no proof of the sender's key; the paper reports all claims verified.

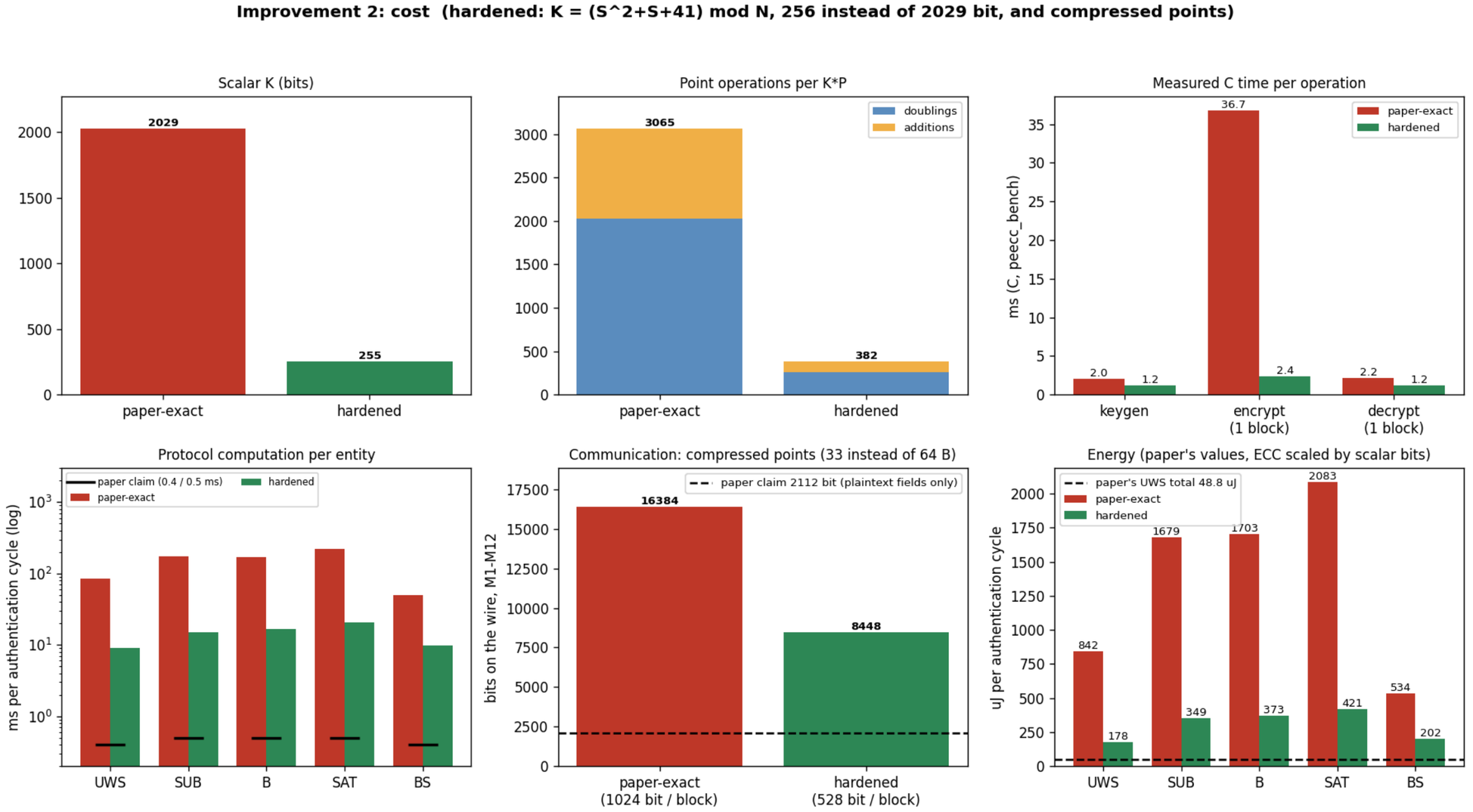
Results - Security Improvement



Still open in both modes: initiator impersonation (M1 has no proof of the private key) and a delay of exactly one 8-bit timestamp period (12.8 s).

Hardened mode detects 9 of 11 attacks (paper-exact 7 of 11); the keyless eavesdropper reads 0 of 12 messages instead of 12 of 12.

Results - Cost and Communication



K 2029 -> 255 bits, ~3065 -> ~380 point operations, encryption ~8x faster, UWS energy 842 -> 178 uJ, bits on the wire 16,384 -> 8,448 (-48 %).

Performance Comparison

Cost (one authentication)

Metric	Paper-exact [18]	Hardened	Change
Key number K	2029 bit	256 bit	8x smaller
Encrypt one block (C)	36.7 ms	2.4 ms	15.2x faster
UWS computation / cycle	86.0 ms	9.0 ms	9.6x less
UWS energy / cycle	842 uJ	178 uJ	4.7x less
M1-M12 on the wire	16,384 bit	8,448 bit	-48 %
End-to-end latency	6.73 s	5.31 s	

Paper claims vs measured (paper-exact / hardened)

Claim	Paper	Measured
Communication	2112 bit	16,384 / 8,448 bit on the wire
Computation / entity	0.4-0.5 ms	86 / 9.0 ms (UWS)
UWS energy	48.8 uJ	842 / 178 uJ
Scyther	all verified	23 Ok, 35 Fail

Security

Metric	Paper-exact [18]	Hardened	Change
Eavesdropper reads M1-M12	12 of 12	0 of 12	fixed
Attacks detected	7 of 11	9 of 11	+2
Same message twice	same ciphertext	different	unlinkable
Off-curve points	accepted	rejected	validated
Decryption correct	1000 / 1000	1000 / 1000	no loss

Measured with peecc_bench and simulate.py --all on this PC (seed 1); times vary with load. Both modes run the same protocol messages on secp256r1. Full scorecard: results/claims_scorecard.png.

Results, Limitation and Future Work

1. Results

- PEECC [18] in C (GMP, secp256r1) and the full UWC protocol; 27 self-tests and 1000 round trips pass.
- Paper-exact PEECC is not confidential: K is public, a keyless attacker reads 12 of 12 protocol messages.
- Hardened PEECC blocks it (0 of 12), with ~8x faster encryption and 48 % fewer bits on the wire.

2. Limitations

- Initiator impersonation and a delay of exactly 12.8 s (8-bit timestamp wrap) are not detected in either mode.
- No integrity tag, not constant-time; the paper's 0.4 ms and 2112-bit claims are not reached.

3. Future Work

- Signature or MAC proving the sender in M1, wider timestamps, faster curve arithmetic.

Conclusion

The pentatope construction can be kept and made confidential by drawing its input at random; the protocol around it still needs sender authentication.

References

1. C. Rupa et al. A Novel and Robust Authentication Protocol for Secure Underwater Communication Systems. IEEE Internet of Things Journal, vol. 12, no. 22, 2025.

2. D. Jayakumari, C. Rupa, V. Nikhila. Pentatope Based Elliptic Curve Encryption for Privacy and Protection of Multimedia Data. (reference [18])

3. Certicom Research. SEC 2: Recommended Elliptic Curve Domain Parameters (secp256r1), 2010; SEC 1: Elliptic Curve Cryptography, 2009.

4. N. Koblitz. Elliptic Curve Cryptosystems. Mathematics of Computation, vol. 48, 1987.

5. C. Cremers. The Scyther Tool: Verification, Falsification, and Analysis of Security Protocols. CAV 2008.

6. The GNU Multiple Precision Arithmetic Library (GMP), gmplib.org.

7. Codebase: https://github.com/D3athSkulll/Auth-Protocol-for-Secure-UWCN

THANK YOU